

## 1. Student Details :

- **Name:** Vikas
- **Roll Number:** 23f3001800
- **Course:** Application Development II (MAD-2), Jan 2026

## Project Title : CampusHire — From Classroom to Career

### Problem Statement :

Institutes require efficient systems to manage campus recruitment activities involving companies and students. Currently, many institutes rely on spreadsheets, emails, or manual coordination, which makes it difficult to manage company approvals, placement drives, student registrations, and application tracking.

You are required to build a Placement Portal Application (PPA) web application that allows Admin (Institute), Companies, and Students to interact with the system based on their roles.

### Approach :

I built CampusHire as a modern decoupled application. The backend strictly serves data via REST APIs, while the frontend handles all the UI routing and state management, resulting in a fast and seamless user experience.

- **Backend** : Built an API-first backend using Flask and Flask-RESTful. It handles token-based authentication via Flask-Security-Too and manages the core business logic, like validating if a student meets the criteria before allowing an application to a drive.
- **Frontend** : Developed a Single Page Application (SPA) using Vue 3 and Pinia. It dynamically renders dashboards based on the user's role (Admin, Company, or Student) and uses Axios to fetch data from the backend APIs asynchronously.
- **Background Jobs** : Integrated Celery with Redis as the message broker. This handles heavy lifting outside the main request cycle, specifically: sending daily deadline reminders to students, generating the monthly HTML activity report for the Admin, and processing user-triggered CSV application exports.
- **Performance Optimization** : Implemented Redis caching for frequently accessed, read-heavy API endpoints to reduce database load. Background jobs ensure the UI never freezes while waiting for email servers or file generation.

### 3. AI / LLM Usage Declaration :

AI was primarily used to assist with the structural boilerplate of the API and Vue components, as well as debugging complex asynchronous tasks

| Component / Tool                 | Function                                                | Usage | Notes                                    |
|----------------------------------|---------------------------------------------------------|-------|------------------------------------------|
| <b>Vue.js (Core + Router)</b>    | Reactive SPA frontend development, routing.             | 18% ▾ | Central SPA framework.                   |
| <b>Flask (App + API)</b>         | Backend logic, serving REST APIs.                       | 18% ▾ | Exclusively for REST API.                |
| <b>SQLite + SQLAlchemy</b>       | Data persistence, relational ORM.                       | 12% ▾ | Standard ORM implementation.             |
| <b>Auth (Security-Tool)</b>      | Login/logout, token-based role access control.          | 10% ▾ | Focus on token security.                 |
| <b>Bootstrap / CSS</b>           | Styling, layout, responsive UI.                         | 25% ▾ | Used for responsive design.              |
| <b>User Logic (Applications)</b> | Core business flow, application management.             | 7% ▾  | Business logic via frontend/API.         |
| <b>Axios / Fetch</b>             | Managing asynchronous API calls/data fetching.          | 12% ▾ | Facilitates async state/data retrieval.  |
| <b>Admin CRUD Logic</b>          | Frontend interfaces for user/drive/company CRUD.        | 5% ▾  | Administrative management tools.         |
| <b>APIs / Integration</b>        | Structuring REST endpoint data communication.           | 10% ▾ | Focus on Blueprint/resource structuring. |
| <b>HTML + Jinja2</b>             | Minimal server-side template rendering for entry point. | 25% ▾ | Only for main entry page.                |

## Database Design (~ 50 words) :

The schema utilizes a relational SQLite database via SQLAlchemy. It anchors on a unified **User** table for authentication, extending into distinct **Student** and **Company** profiles. An **Application** junction table securely maps students to company-created **PlacementDrives**, enabling real-time status tracking and maintaining comprehensive historical placement records.

## What I implemented :

I built a complete, role-based placement portal. Admins can govern the platform and view stats; Companies can post drives and manage applicant pipelines (including scheduling interviews and uploading offer letters); Students can track their job hunts, upload resumes, and view their placement history. I also integrated Redis and Celery to handle async CSV exports and scheduled email reports.

## 4. Technologies and Frameworks :

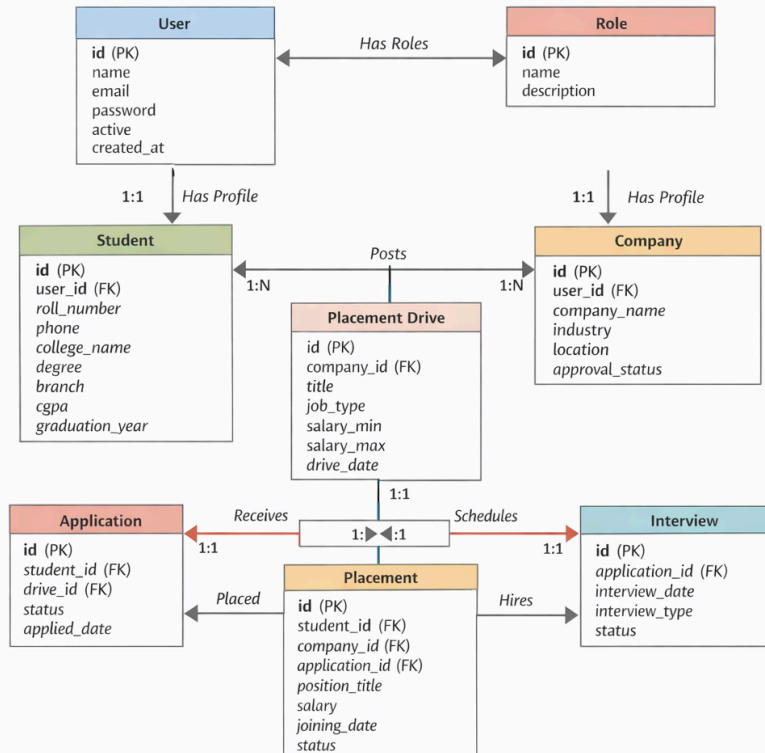
- **Backend:** Flask 3.x, Flask-RESTful
- **Authentication:** Flask-Security-Too (Token-based)
- **Frontend:** Vue 3 (Composition API), Pinia, Vue Router, Axios
- **Database:** SQLite, SQLAlchemy ORM
- **Async & Caching:** Redis, Celery 5.x, Flask-Caching
- **Styling:** Bootstrap 5

## 5. Database Schema :

- **User:** Handles authentication and role identification (**email**, **password**, **active**).
- **Student:** Extends User; stores academic details (CGPA, branch, resume file path).
- **Company:** Extends User; stores organization details (name, HR contact).
- **PlacementDrive:** Created by Companies, requires Admin approval. Stores job descriptions, eligibility criteria, and deadlines.
- **Application:** Tracks the many-to-many relationship between Students and Drives, including current progression status (Applied, Shortlisted, Selected)

## ER Diagram :

## ER Diagram – CampusHire Placement System



## 6. API Resource Endpoints :

Note: All endpoints are prefixed with */api*.

| Role             | Method    | Path                                                             | Description          |
|------------------|-----------|------------------------------------------------------------------|----------------------|
| <b>Student</b> ▾ | GET/PUT ▾ | <i>/student/&lt;id&gt;</i>                                       | Fetch/update profile |
| <b>Student</b> ▾ | GET ▾     | <i>/student/&lt;id&gt;/ap<br/>plications</i>                     | View applied drives  |
| <b>Student</b> ▾ | POST ▾    | <i>/student/&lt;id&gt;/ap<br/>ply/&lt;drive_id&gt;</i>           | Apply to drive       |
| <b>Student</b> ▾ | DELETE ▾  | <i>/student/&lt;id&gt;/ap<br/>plications/&lt;app_<br/>id&gt;</i> | Withdraw application |
| <b>Student</b> ▾ | POST ▾    | <i>/student/&lt;id&gt;/ex</i>                                    | Export CSV history   |

|           |            |                                                       |                         |
|-----------|------------|-------------------------------------------------------|-------------------------|
|           |            | port/csv                                              |                         |
| Company ▾ | POST/PUT ▾ | /company/<id>                                         | Create/update profile   |
| Company ▾ | GET/POST ▾ | /company/<company_id>/drives                          | List/create drives      |
| Company ▾ | GET ▾      | /company/<company_id>/drives/<drive_id>/applicants    | View drive applicants   |
| Company ▾ | POST ▾     | /company/<company_id>/applications/<app_id>/interview | Schedule interview      |
| Company ▾ | PATCH ▾    | /company/<company_id>/applications/<app_id>/selection | Update selection status |
| Admin ▾   | GET ▾      | /admin/stats                                          | Fetch dashboard stats   |
| Admin ▾   | GET ▾      | /admin/search                                         | Global search           |
| Admin ▾   | GET ▾      | /admin/applications                                   | View all applications   |
| Shared ▾  | GET ▾      | /drives                                               | Fetch approved drives   |
| Shared ▾  | GET ▾      | /uploads/resumes/<filename>                           | Serve user resumes      |
| Shared ▾  | POST ▾     | /upload-offer                                         | Upload offer letters    |

## **System Architecture :**

CampusHire utilizes a decoupled Client-Server architecture. The Vue 3 client runs in the browser, requesting and submitting JSON data via Axios to the Flask REST API. The API reads and writes to a SQLite database. To ensure the UI remains responsive, heavy tasks (like CSV generation and bulk emails) are passed to a Redis message broker and executed asynchronously by Celery workers.

## **Core Features Implemented :**

- Multi-role JWT-based authentication and restricted dashboards (Admin, Student, Company).
- Admin approval workflows for new company registrations and placement drives.
- Automated drive eligibility filtering for students based on CGPA and branch.
- File handling for student resumes and company offer letters.
- Celery-powered scheduled jobs (Daily deadline reminders, Monthly placement reports).
- User-triggered asynchronous CSV data export for student application history.

## **9. Learning Outcomes :**

A major learning outcome was building strict REST APIs to serve a separate frontend application. Managing global state with Pinia taught me how to keep the frontend UI fully in sync without forcing the user to refresh the page. Setting up Celery with Redis also provided highly practical experience in handling long-running background processes so the main application stays fast, responsive, and professional.